explicitly, you may find that some of your types are unexpectedly marked optional, which can lead to some hard-to-debug errors.

TIP Keep NRTs enabled for your minimal API endpoints wherever possible. If you can't use them for your whole project, consider enabling them selectively in Program.cs (or wherever you add your endpoints) by adding `#nullable enable` to the top of the file.

The good news is that ASP.NET Core includes several analyzers built into the compiler to catch configuration problems like the ones described in this section. If you have an optional route parameter but forget to mark the corresponding handler parameter as optional, for example, integrated development environments (IDEs) such as Visual Studio will show a hint, as shown in figure 7.7, and you'll get a build warning. You can read more about the built-in analyzers at http://mng.bz/4DMV.

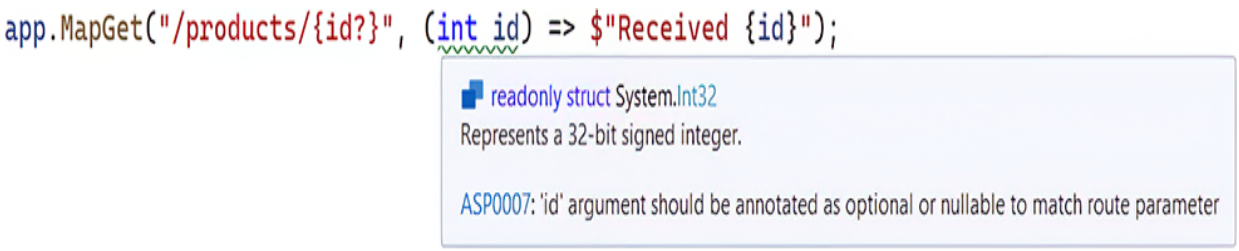


Figure 7.7 Visual Studio and other IDEs use analyzers to detect potential problems with mismatched optionality.

Making your handler parameters optional is one of the approaches you can take, whether they're bound to route parameters, headers, or the query string. Alternatively, you can provide a default value for the parameter as part of the method signature. You can't provide default values for parameters in lambda functions in C# 11,[2] so the following listing shows how to use a local function instead.